Guide de Migration

race-simulation-rl

Stack Moderne : Python 3.11 · PyTorch · Stable-Baselines3 · 0% TensorFlow

Composant	Ancienne version	Nouvelle version	Statut
Python	3.8	3.11	Supporté jusqu'en 2027
numpy	1.23.5	1.26.4	Dernière numpy 1.x stable
tensorflow	2.3.0	■ Supprimé	Remplacé par PyTorch
tf-agents	ancienne	■ Supprimé	Remplacé par SB3
torch	non épinglé	2.4.1	GPU RTX 30/40xx ✓
stable-baselines3	■ absent	2.3.2	DQN, PPO, SAC...
gymnasium	■ absent	0.29.1	Remplace gym
onnxruntime	■ absent	1.19.2	Inférence (remplace TFLite)
scipy	1.7.3	1.13.1	
cvxpy	1.2.1	1.5.3	
matplotlib	3.5.3	3.9.2	

■■ TensorFlow est entièrement supprimé. L'inférence des modèles .tflite existants est remplacée par onnxruntime après une conversion one-shot des modèles.

0. Architecture du projet

Le projet contient 5 composants principaux. Les changements touchent uniquement les composants RL training et VSE — le simulateur de course lui-même n'est pas modifié.

Composant	Fichiers clés	Modifié ?
Simulateur de course	racesim/src/race.py, track.py...	■ Non
VSE Supervisé	racesim/src/vse_supervised.py	═■ Oui (TF→ONNX)
VSE Reinforcement	racesim/src/vse_reinforcement.py	═■ Oui (TF→ONNX)
Preprocessing	machine_learning/src/preprocessor.py	═■ 1 ligne
RL Training	machine_learning_rl_training/src/*	═■ Interface gymnasium
Script principal	main_train_rl_agent_dqn.py	■ Remplacé par main_train_sb3.py

1. requirements.txt

```
- pip<24.1
- setuptools<65.0.0
- wheel<0.40.0
- numpy==1.23.5
- scipy==1.7.3
- cvxpy==1.2.1
- matplotlib==3.5.3
- protobuf==3.20.3
- tensorflow==2.3.0
- torch
- scikit-learn
- pandas
- tqdm
+ numpy==1.26.4
+ scipy==1.13.1
+ cvxpy==1.5.3
+ matplotlib==3.9.2
+ pandas==2.2.3
+ scikit-learn==1.5.2
+ tqdm==4.66.5
+ # Deep Learning — training RL
+ torch==2.4.1
+ stable-baselines3==2.3.2
+ gymnasium==0.29.1
+ # Inference modèles existants (remplace tensorflow)
+ onnxruntime==1.19.2
```

■■ TensorFlow, tf-agents, protobuf sont entièrement supprimés.

2. Conversion des modèles TFLite → ONNX (à faire une seule fois)

Les modèles pré-entraînés du projet sont au format .tflite (TensorFlow Lite). On les convertit en .onnx une seule fois, puis onnxruntime remplace tf.lite.Interpreter partout. Après conversion, TensorFlow n'est plus nécessaire.

2a. Installer les outils de conversion (hors sandbox, avec réseau)

```
pip install tensorflow-cpu==2.17.0 tf2onnx==1.16.1
```

2b. Script de conversion — convert_tflite_to_onnx.py

```
"""
Convertit tous les fichiers .tflite du projet en .onnx.
À exécuter une seule fois depuis la racine du projet.
"""
import os, glob, subprocess

tflite_files = glob.glob("racesim/input/vse/**/*.tflite", recursive=True)
tflite_files += glob.glob("machine_learning_rl_training/output/*.tflite")

for tflite_path in tflite_files:
    onnx_path = tflite_path.replace(".tflite", ".onnx")
    print(f"Converting: {tflite_path} -> {onnx_path}")
    subprocess.run([
        "python", "-m", "tf2onnx.convert",
        "--tflite", tflite_path,
        "--output", onnx_path,
        "--opset", "17",
    ], check=True)
    print(f"  Done: {onnx_path}")

print(f"\nConverted {len(tflite_files)} models.")
```

■ Exécuter depuis la racine du projet : python convert_tflite_to_onnx.py

3. racesim/src/vse_reinforcement.py

Ce fichier charge et fait tourner le Q-network DQN entraîné par RL. Il utilise tf.lite.Interpreter pour l'inférence. On le remplace par onnxruntime.

3a. Changement de l'import

```
- import tensorflow as tf
+ import onnxruntime as ort
```

3b. Constructeur — chargement du modèle

```
- self.nn_model = {"interpreter": tf.lite.Interpreter(model_path=nnmodel_path)}
- self.nn_model["interpreter"].allocate_tensors()
- self.nn_model["input_index"]  = self.nn_model["interpreter"].get_input_details()[0]["index"]
- self.nn_model["output_index"] = self.nn_model["interpreter"].get_output_details()[0]["index"]
- self.no_actions = int(self.nn_model["interpreter"].get_output_details()[0]["shape"][1])

+ # nnmodel_path doit pointer vers le .onnx (au lieu de .tflite)
+ self.nn_model = ort.InferenceSession(nnmodel_path,
+                     providers=["CPUExecutionProvider"])
+ self.nn_model_input_name  = self.nn_model.get_inputs()[0].name
+ self.no_actions = self.nn_model.get_outputs()[0].shape[1]
```

3c. Méthode make_decision — inférence

```
  # AVANT — inférence TFLite
```

```
- self.nn_model["interpreter"].set_tensor(
-     self.nn_model["input_index"],
-     np.expand_dims(self.X_conv[idx_driver], axis=0))
- self.nn_model["interpreter"].invoke()
- action_q_vals[idx_driver] = self.nn_model["interpreter"].get_tensor(
-     self.nn_model["output_index"])[0]
```

```
  # APRÈS — inférence ONNX
```

```
+ inp = np.expand_dims(self.X_conv[idx_driver], axis=0)  # shape (1, obs_size)
+ action_q_vals[idx_driver] = self.nn_model.run(
+     None, {self.nn_model_input_name: inp})[0][0]
```

3d. Mise à jour du chemin dans les fichiers .ini

Dans tous les fichiers racesim/input/vse/*.ini, changer l'extension :

```
- nn_reinforcement = nn_reinforcement_Spa_2019.tflite
+ nn_reinforcement = nn_reinforcement_Spa_2019.onnx
```

4. racesim/src/vse_supervised.py

Ce fichier gère deux réseaux (tirechange + compound choice). Même logique que vse_reinforcement.

4a. Import

```
- import tensorflow as tf
+ import onnxruntime as ort
```

4b. Constructeur

```
- self.nnmodel_cc = {"interpreter": tf.lite.Interpreter(model_path=nnmodel_cc_path)}
- self.nnmodel_tc = {"interpreter": tf.lite.Interpreter(model_path=nnmodel_tc_path)}
- self.nnmodel_cc["interpreter"].allocate_tensors()
- self.nnmodel_cc["input_index"]  = self.nnmodel_cc["interpreter"].get_input_details()[0]["index"]
- self.nnmodel_cc["output_index"] = self.nnmodel_cc["interpreter"].get_output_details()[0]["index"]
- self.nnmodel_tc["interpreter"].allocate_tensors()
- self.nnmodel_tc["input_index"]  = self.nnmodel_tc["interpreter"].get_input_details()[0]["index"]
- self.nnmodel_tc["output_index"] = self.nnmodel_tc["interpreter"].get_output_details()[0]["index"]
- self.no_timesteps_tc = self.nnmodel_tc["interpreter"].get_input_details()[0]["shape"][1]
```

```
+ self.nnmodel_cc = ort.InferenceSession(nnmodel_cc_path,
+                     providers=["CPUExecutionProvider"])
+ self.nnmodel_tc = ort.InferenceSession(nnmodel_tc_path,
+                     providers=["CPUExecutionProvider"])
+ self.nnmodel_cc_input = self.nnmodel_cc.get_inputs()[0].name
+ self.nnmodel_tc_input = self.nnmodel_tc.get_inputs()[0].name
+ self.no_timesteps_tc  = self.nnmodel_tc.get_inputs()[0].shape[1]
```

4c. Méthode make_decision — inférence TC (tirechange)

```
- self.nnmodel_tc["interpreter"].set_tensor(
-     self.nnmodel_tc["input_index"],
-     np.expand_dims(self.X_conv_tc[idx_driver], axis=0))
- self.nnmodel_tc["interpreter"].invoke()
- pitstop_probs[idx_driver] = self.nnmodel_tc["interpreter"].get_tensor(
-     self.nnmodel_tc["output_index"])
```

```
+ inp = np.expand_dims(self.X_conv_tc[idx_driver], axis=0)
+ pitstop_probs[idx_driver] = self.nnmodel_tc.run(
+     None, {self.nnmodel_tc_input: inp})[0][0, 0]
```

4d. Méthode make_decision — inférence CC (compound choice)

```
- self.nnmodel_cc["interpreter"].set_tensor(
-     self.nnmodel_cc["input_index"],
-     np.expand_dims(self.X_conv_cc[idx_abs], axis=0))
- self.nnmodel_cc["interpreter"].invoke()
- rel_compound_probs[idx_rel] = self.nnmodel_cc["interpreter"].get_tensor(
-     self.nnmodel_cc["output_index"])
```

```
+ inp = np.expand_dims(self.X_conv_cc[idx_abs], axis=0)
+ rel_compound_probs[idx_rel] = self.nnmodel_cc.run(
+     None, {self.nnmodel_cc_input: inp})[0][0]
```

5. machine_learning/src/preprocessor.py

np.bool a été supprimé dans numpy 1.24. Une seule ligne à corriger.

Ligne 232

```
- .astype(np.bool))[1]
+ .astype(bool))[1]
```

6. Nouveau fichier : rl_environment_gymnasium.py

Remplace rl_environment_single_agent.py (tf-agents) par l'interface gymnasium. La logique de simulation est identique — seule l'interface change.

tf-agents (ancienne)	gymnasium (nouvelle)
tf_agents.environments.PyEnvironment	gymnasium.Env
_reset() → TimeStep	reset() → (obs, info)
_step() → TimeStep	step() → (obs, reward, terminated, truncated, info)
tf_agents.specs.BoundedArraySpec	spaces.Box / spaces.Discrete
TFPyEnvironment wrapper	Pas de wrapper nécessaire

Interface reset() et step()

```
  # ANCIENNE interface (tf-agents)
- def _reset(self) -> tf_agents.trajectories.time_step.TimeStep:
-     ...
-     return tf_agents.trajectories.time_step.restart(observation=observation)

- def _step(self, action) -> tf_agents.trajectories.time_step.TimeStep:
-     ...
-     return tf_agents.trajectories.time_step.transition(
-         observation=obs, reward=reward, discount=1.0)


  # NOUVELLE interface (gymnasium)
+ def reset(self, seed=None, options=None):
+     super().reset(seed=seed)
+     ...  # même logique
+     return obs, {}            # tuple (observation, info dict)

+ def step(self, action):
+     ...  # même logique
+     return obs, float(reward), terminated, False, {}
+     #                          ^^^^^^^^^^  ^^^^^^^^^
+     #                          terminated  truncated (tjrs False ici)
```

Espaces d'observation et d'action

```
  # ANCIENNE (tf-agents specs)
- self._action_spec = tf_agents.specs.array_spec.BoundedArraySpec(
-     shape=(), dtype=np.int32, minimum=0, maximum=len(compounds), name='action')
- self._observation_spec = tf_agents.specs.array_spec.BoundedArraySpec(
-     shape=(obs_size,), dtype=np.float32, minimum=0.0, maximum=1.0)


  # NOUVELLE (gymnasium spaces)
+ self.action_space = spaces.Discrete(len(compounds) + 1)
+ self.observation_space = spaces.Box(
+     low=0.0, high=1.0, shape=(obs_size,), dtype=np.float32)
```

7. machine_learning_rl_training/src/rl_evaluate_policy.py

Import matplotlib manquant (crash au post-training) :

```
  import numpy as np
- import tensorflow as tf
- import tf_agents.trajectories
+ import matplotlib.pyplot as plt   # ← manquant dans le fichier original
```

8. Nouveau script : main_train_sb3.py

Remplace main_train_rl_agent_dqn.py. Compatible PyTorch + Stable-Baselines3.

```
from machine_learning_rl_training.src.rl_environment_gymnasium import RaceSimulationGymnasium
from stable_baselines3 import DQN  # Changer en PPO ou SAC facilement
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.env_checker import check_env
import racesim, os, numpy as np

# ■■ Paramètres ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
race              = "Spa_2019"
controlled_driver = "HAM"
vse_others        = "supervised"
mcs_pars_file     = "pars_mcs.ini"
num_timesteps     = 250_000
output_dir        = "machine_learning_rl_training/output"
os.makedirs(output_dir, exist_ok=True)

# ■■ Vérification course humide ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pars = racesim.src.import_pars.import_pars(use_print=False, use_vse=False,
    race_pars_file=f"pars_{race}.ini", mcs_pars_file=mcs_pars_file)[0]
for drv in pars["driver_pars"]:
    if any(s[1] in ["I","W"] for s in pars["driver_pars"][drv]["strategy_info"]):
        raise RuntimeError(f"Course humide : {race}")

# ■■ Environnements ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
kwargs = dict(race_pars_file=f"pars_{race}.ini", mcs_pars_file=mcs_pars_file,
              vse_type=vse_others, use_prob_infl=True,
              create_rand_events=True, controlled_driver=controlled_driver)
train_env = RaceSimulationGymnasium(**kwargs)
eval_env  = RaceSimulationGymnasium(**kwargs)
check_env(train_env, warn=True)  # valide le format de l'env

# ■■ Agent DQN (équivalent à l'ancienne config) ■■■■■■■■■■■■■■■■■■■■■■■■
model = DQN(
    policy="MlpPolicy", env=train_env,
    learning_rate=1e-3,
    buffer_size=200_000,          # = replay_buffer_max_length
    learning_starts=200,          # = initial_collect_steps
    batch_size=64,
    gamma=0.97,                   # était 1.0 → 0.97 recommandé
```

```
    train_freq=1,
    target_update_interval=1,
    policy_kwargs={"net_arch": [64, 64]},  # = fc_layer_params
    exploration_fraction=0.5,
    exploration_final_eps=0.01,
    verbose=1,
    tensorboard_log=os.path.join(output_dir, "tensorboard"),
    device="auto",               # GPU si dispo, sinon CPU
)

# ■■ Pour passer à PPO, remplacer les 2 lignes DQN par :
# from stable_baselines3 import PPO
# model = PPO("MlpPolicy", train_env,
#             policy_kwargs={"net_arch": [64, 64]}, verbose=1)

# ■■ Entraînement ■■■
```

9. Récapitulatif — Tous les changements

Fichier	Type de changement	Lignes modifiées
requirements.txt	Stack mise à jour, TF supprimé	Tout
machine_learning/src/preprocessor.py	np.bool → bool	L.232
racesim/src/vse_reinforcement.py	tf.lite → onnxruntime	~10 lignes
racesim/src/vse_supervised.py	tf.lite → onnxruntime	~15 lignes
rl_environment_single_agent.py	np.float → float	L.557
rl_evaluate_policy.py	Ajouter import matplotlib	L.~21
rl_environment_gymnasium.py	■ Nouveau fichier (wrapper gym)	~160 lignes
main_train_sb3.py	■ Nouveau script (SB3)	~80 lignes
convert_tflite_to_onnx.py	■ Script one-shot conversion	~20 lignes
racesim/src/* (reste)	■ Aucun changement	—

Ordre d'exécution recommandé

Étape 1 — Mettre à jour requirements.txt et installer le nouvel environnement

```
$ pip install -r requirements.txt
```

Étape 2 — Convertir tous les modèles .tflite → .onnx (nécessite accès réseau pour installer tf2onnx)

```
$ python convert_tflite_to_onnx.py
```

Étape 3 — Appliquer les 3 corrections dans le code existant

```
$ preprocessor.py, rl_environment_single_agent.py, rl_evaluate_policy.py
```

Étape 4 — Mettre à jour racesim/src/vse_supervised.py et vse_reinforcement.py

```
$ tf.lite → onnxruntime (voir sections 3 et 4)
```

Étape 5 — Créer rl_environment_gymnasium.py

```
$ Nouveau wrapper gymnasium (voir section 6)
```

Étape 6 — Lancer l'entraînement avec le nouveau script

```
$ python main_train_sb3.py
```

Étape 7 — (Optionnel) Visualiser avec TensorBoard

```
$ tensorboard --logdir machine_learning_rl_training/output/tensorboard
```

■■ L'étape 2 (conversion) nécessite tensorflow-cpu==2.17.0 et tf2onnx installés temporairement. Une fois la conversion faite, TensorFlow peut être désinstallé.

■ Après toutes ces étapes, le projet tourne entièrement sur PyTorch + Stable-Baselines3. TensorFlow n'est plus nécessaire.